SOFTWARE ENGINEERING LAB

EXPERIMENT NO. 8

Modular Design of a Software System

kvstore

A Distributed, Fault-Tolerant, Transactional Key–Value Store

High Cohesion • **Low Coupling**

Clear Interfaces • Layered Dependencies

Name	Neha Nupur
Roll Number	20251651064
Programme	MCA, Second Semester
Subject Code	CS546
Subject	Software Engineering
Academic Session	2026

Prepared with reference to the Lab 8 student submission template, the supplied modular-design reference PDF, and the `kvstore` Software Requirements Specification.

Contents

1 Aim and Project Overview **2**
1.1 Aim . . . 2
1.2 Project overview . . . 2
1.3 Why modular design is appropriate . . . 2

2 Main Functional Requirements **2**

3 Module Identification **3**

4 Modular Design Diagram **4**
4.1 How to read the diagram . . . 4

5 Interfaces Between Modules **5**
5.1 Interface 1: Transaction API . . . 5
5.2 Interface 2: Wire Envelope . . . 5
5.3 Interface 3: StatsProvider . . . 6
5.4 Interface 4: Key Routing . . . 6

6 Cohesion: Why Do These Tasks Belong Together? **6**
6.1 Two explicit cohesion decisions . . . 7

7 Coupling: How Were Dependencies Reduced? **7**
7.1 Two explicit coupling decisions . . . 7

8 Tutorial Numericals and Dependency Analysis **8**
8.1 Numerical 1: Count module connections and average coupling . . . 8
8.2 Numerical 2: Module instability . . . 9

9 Design Review Checklist **10**

10 Result **10**

11 References **10**

1 Aim and Project Overview

1.1 Aim

To divide the `kvstore` software system into meaningful modules, define the interfaces through which these modules communicate, and justify design decisions that promote high cohesion and low coupling.

1.2 Project overview

`kvstore` is a distributed, replicated, transactional key–value store implemented in Go using the Go standard library. It supports Put, Get, Delete, and atomic transactions, while using Raft consensus for replicated ordering, MVCC for snapshot isolation, a checksummed write-ahead log for durability, and a hybrid logical clock for timestamp ordering. The system also provides a network transport layer, client library, metrics, a REST API and live dashboard, operational intelligence, and optional sharding and TLS. The design separates these responsibilities into packages so that each part can be developed, tested, and maintained independently.

Design takeaway

Design objective. Keep transaction correctness and durability in the core data path, expose stable interfaces at module boundaries, and prevent operational intelligence or user-interface code from depending directly on Raft and WAL internals.

1.3 Why modular design is appropriate

- **Understandability:** each package owns one well-defined responsibility.
- **Independent testing:** WAL replay, clock ordering, metrics, and statistical analysis can be tested without running a complete cluster.
- **Fault isolation:** defects can be localised to a responsibility such as consensus, storage, or transport.
- **Change containment:** a change to the dashboard or AI analysis should not require changes to the WAL or Raft internals.
- **Dependency control:** lower-level primitives do not import higher-level application packages.

2 Main Functional Requirements

The following requirements are distilled from the supplied `kvstore` SRS and the Lab 8 reference document. They describe the capabilities that the module structure must support.

[redacted]	Requirement	Description
R1	Key–value operations and transactions	Store, retrieve, and delete values; group multiple operations into atomic transactions.
R2	Fault-tolerant replication	Replicate operations through Raft, elect a leader, and continue safely when a minority of nodes fails.
R3	Transaction isolation	Provide snapshot-isolated reads and deterministic first-committer-wins conflict detection.
R4	Durability and recovery	Append checksummed records to the write-ahead log, flush them durably, and replay valid records after a crash.
R5	Deterministic timestamps	Generate hybrid logical clock timestamps and use them to order versions and transaction events.

R6	Observability	Collect counters, gauges, and histograms; export metrics in Prometheus and JSON formats and expose a live dashboard.
R7	Operational intelligence	Analyse metrics for anomalies, correlations, trends, leadership advice, tuning recommendations, and natural-language queries.
R8	Horizontal scaling and security	Route keys to shards, support the specified cross-shard transaction path, use client redirection, and optionally secure transport with TLS/mTLS.

3 Module Identification

The responsibilities are grouped by the principle that one module should have one primary reason to change. The module names and package paths below follow the supplied reference design and the `kvstore` SRS. Command-line entry points are shown as front-end modules; supporting packages are included where they have a distinct responsibility.

Module	Package / Location	One Main Responsibility	Req.
CLI Client	`cmd/kvctl`	Provide command-line access to read, write, delete, and inspect cluster status.	R1,R2
Web Layer	`internal/httpapi`	Expose REST endpoints, Server-Sent Events, dashboard assets, and composition/wiring adapters.	R1,R6,R7
Sharding Server	`cmd/kvgateway`	Provide one logical gateway address in front of independent shard clusters.	R8
Transaction Engine	`internal/storage`	Coordinate Begin, Get, Put, Delete, Rollback, and Commit; apply committed operations to the state machine.	R1,R3
Operational AI	`internal/ai`	Analyse supplied statistics for anomaly detection, correlation, forecasting, advice, tuning, and queries.	R7
Routing Ring	`internal/shard`	Deterministically map each key to its responsible shard using rendezvous hashing.	R8
Consensus	`internal/raft`	Manage Raft elections, replication, ordered log commitment, snapshots, and membership.	R2,R4
Version Store	`internal/mvcc`	Maintain version chains, snapshot reads, write-conflict checks, and version garbage collection.	R1,R3,R5
Hybrid Clock	`internal/hlc`	Generate and compare hybrid logical clock timestamps for deterministic ordering.	R5
Write-Ahead Log	`internal/wal`	Append opaque records, checksum and fsync them, and replay valid records after restart.	R4
Wire Protocol	`internal/transport`	Encode and exchange request/reply envelopes between clients and nodes; return leader hints.	R2,R8
Client Library	`internal/client`	Discover the leader, follow NotLeader hints, and retry requests across candidate nodes.	R1,R2
Metrics Registry	`internal/metrics`	Record counters, gauges, and histograms and export Prometheus/JSON metrics.	R6,R7
Security Helpers	`internal/tlsutil`	Generate development certificates and load TLS or mutual-TLS credentials.	R8

Fault Injection	`internal/fault`	Simulate partitions, delays, message loss, kills, and restarts for tests only.	R2,R4

Scope note: The main dependency diagram below emphasises the 13 production modules and connections enumerated in the supplied reference's tutorial numericals. Security Helpers and Fault Injection are supporting modules: TLS utilities are optional, and Fault Injection is test-only, so neither is included in the production connection count.

4 Modular Design Diagram

The diagram uses four layers. An arrow $A \rightarrow B$ means that module A depends on or calls services from module B. The intended dependency direction is downward: front ends and composition code may depend on lower-level modules, but primitives must not import application or presentation layers.

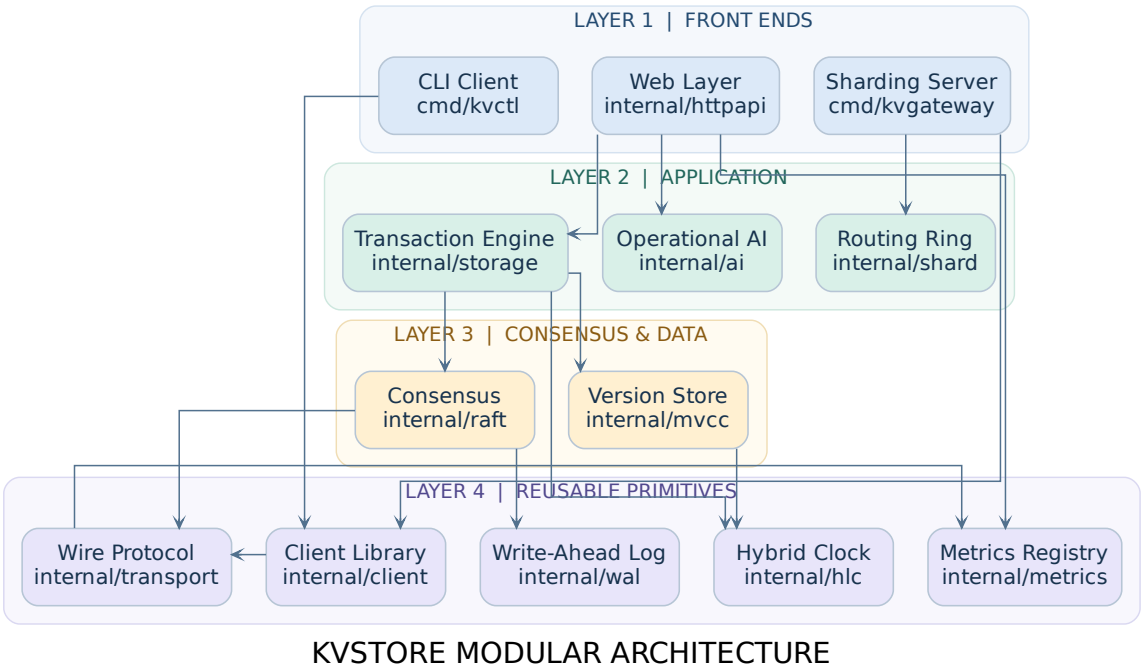


Figure 1: Custom module dependency diagram for `kvstore`. Arrows point from a dependent module to the module it uses. The diagram shows the 13 production modules used in the coupling numericals.

Method note

Arrow convention. An arrow $A \rightarrow B$ means that module A depends on or calls module B. It does not mean that runtime data necessarily travels in the same direction. The 14 direct dependencies are enumerated explicitly in the numerical analysis section so the diagram can be checked against the calculation.

4.1 How to read the diagram

- **Layer 1 — Front Ends:** command-line, HTTP/dashboard, and gateway entry points.

- **Layer 2 — Application:** transactional orchestration, statistical operational intelligence, and shard selection.
- **Layer 3 — Consensus and Data:** Raft ordering and MVCC version management.
- **Layer 4 — Primitives:** reusable clock, WAL, transport, metrics, and client communication support.

5 Interfaces Between Modules

An interface defines an operation, its inputs, its outputs, and the modules on either side. Interfaces keep implementation details private and allow modules to be tested with substitutes or mocks. Four interfaces are documented below, exceeding the template's minimum of three.

5.1 Interface 1: Transaction API

Calling module: Web Layer → **Providing module:** Transaction Engine.

```
type Engine interface {
    Begin(ctx context.Context) (*Txn, error)
}

func (t *Txn) Get(key string) (value []byte, ok bool)
func (t *Txn) Put(key string, value []byte)
func (t *Txn) Delete(key string)
func (t *Txn) Rollback()
func (t *Txn) Commit(ctx context.Context) (hlc.Timestamp, error)
```

Operation	Input	Output	Purpose
Begin	Context	Transaction handle and snapshot	Start a transaction.
Get	Key	Value bytes and found flag	Read the version visible to the transaction.
Put/Delete	Key and optional value	Buffered write-set update	Stage a change before commit.
Commit	Open transaction and context	Commit timestamp or error	Submit changes through consensus and enforce conflict rules.
Rollback	Open transaction	No return value	Discard uncommitted changes.

5.2 Interface 2: Wire Envelope

Calling/providing modules: Client Library or Consensus ↔ Wire Protocol.

```
type Envelope struct {
    Kind byte // RequestVote, AppendEntries, InstallSnapshot,
                    // TimeoutNow, or ClientRequest
    RequestVote *raft.RequestVoteArgs
    AppendEntries *raft.AppendEntriesArgs
    ClientRequest *ClientRequest
    // Snapshot and leadership-transfer payloads are also supported.
}

type EnvelopeReply struct {
    Err string
    NotLeader bool
    LeaderHint string
}
```

Direction	Input	Output / meaning
Request	Structured envelope containing a client operation or Raft message	Message is encoded and sent over a TCP connection.
Reply	Request processed by remote node	One reply contains the result or an error.
Redirect	Request reaches a non-leader	Reply sets `NotLeader` and may provide `LeaderHint`; client retries at that node.

5.3 Interface 3: StatsProvider

Calling module: Web Layer adapter → **Providing module:** Operational AI.

```
type StatsProvider interface {
    MetricsSnapshot() metrics.Snapshot
    NodeObservations() []NodeObservation
    VersionCount() int
}
```

Method	Output	Purpose
MetricsSnapshot	Current counters, gauges, and histogram values	Input for anomaly and trend analysis.
NodeObservations	Per-node latency, errors, replication lag, and queue depth	Input for leadership and load advice.
VersionCount	Number of live MVCC versions	Signal for version-GC pressure and tuning recommendations.

5.4 Interface 4: Key Routing

Calling module: Sharding Server → **Providing module:** Routing Ring.

```
func NewRing(shards []ID) *Ring
func (r *Ring) ShardFor(key string) ID
```

Operation	Input	Output / guarantee
`ShardFor`	Key string	Shard ID responsible for the key. The same key and shard set yield the same result on every machine and after restart.

6 Cohesion: Why Do These Tasks Belong Together?

Cohesion describes how closely related the responsibilities inside one module are. High cohesion means a module has one clear purpose and its functions contribute to that purpose.

Module	Tasks grouped together	Why cohesion is high
Write-Ahead Log	Append opaque records, checksum, fsync, replay after crash.	Every task concerns durable sequential log mechanics. It does not interpret transactions or Raft commands.
Hybrid Clock	Generate, compare, and advance timestamps.	All operations support deterministic event ordering.
Version Store	Maintain versions, snapshot reads, conflict checks, garbage collection.	These operations all implement MVCC and transaction visibility.

Operational AI	Detect anomalies, correlate metrics, forecast trends, advise, tune, answer queries.	All functions analyse operational statistics; it does not own networking, storage, or consensus.
Metrics Registry	Record counters, gauges, histograms and export them.	Collection and presentation of measurements form one observability responsibility.
Web Layer	REST API, dashboard, SSE stream, adapters and system wiring.	These functions form the external access and composition boundary, although cohesion is broader than a single-purpose primitive.

6.1 Two explicit cohesion decisions

C1 — Keep the WAL generic. The WAL stores opaque byte records and performs append, checksum, flush, and replay. It must not parse transaction semantics or decide whether a Raft entry is valid. If it did, a change to consensus meaning would also force changes to low-level disk code, reducing cohesion.

C2 — Keep Operational AI focused on statistics. The AI module receives data through narrow interfaces and performs analysis. It must not directly access the Raft log, storage engine internals, or network transport. This makes it possible to test the algorithms using synthetic metric snapshots without starting a cluster.

7 Coupling: How Were Dependencies Reduced?

Coupling describes the degree of dependency between modules. Low coupling is achieved when modules communicate through small, stable interfaces and do not access one another's private state.

Module connection	Needed?	Reason	How coupling stays low
Operational AI ↔ cluster statistics	Yes	Analysis requires observations and metrics.	Use the small `StatsProvider` interface; AI does not import Raft or storage.
Transaction Engine → Consensus	Yes	Commits must be replicated and ordered.	Consensus accepts opaque operations and applies them through a narrow state-machine boundary.
Version Store → Hybrid Clock	Yes	Versions need comparable timestamps.	Share a small timestamp type rather than clock internals.
Consensus → Wire Protocol	Yes	Nodes must exchange Raft RPCs.	Transport serialises envelopes and does not implement consensus decisions.
Consensus → WAL	Yes	Terms, votes, entries, and snapshots must persist.	WAL treats records as opaque bytes and knows nothing about Raft semantics.
Web Layer → multiple modules	Yes, controlled	The application boundary must wire the system together.	Confine broad dependencies to this composition root; other modules remain narrowly connected.

7.1 Two explicit coupling decisions

D1 — Depend on an interface, not on a concrete cluster. Operational AI uses `StatsProvider`. The Web Layer supplies an adapter. A change to Raft internals therefore does not automatically

require changes to AI algorithms, provided the interface remains stable.

D2 — Consensus handles opaque operations. Raft orders and replicates log entries without understanding key/value semantics. The Transaction Engine interprets committed operations. This avoids coupling consensus to MVCC data structures and makes the consensus module easier to test and reason about.

Controlled exception — Web Layer. The Web Layer intentionally depends on several modules because it is the composition root. This broad dependency is acceptable when it is isolated to one boundary rather than spread throughout the codebase.

8 Tutorial Numericals and Dependency Analysis

The supplied reference counts 14 production dependency connections across 13 drawn modules. Each connection is counted once, in the direction *from the dependent module to the module it depends on.* The fault-injection harness and optional TLS helper are supporting modules and are excluded from this production graph.

8.1 Numerical 1: Count module connections and average coupling

The 14 connections listed in the reference are:

1. CLI Client → Client Library.
2. Sharding Server → Client Library.
3. Sharding Server → Routing Ring.
4. Web Layer → Transaction Engine.
5. Web Layer → Operational AI.
6. Web Layer → Metrics Registry.
7. Transaction Engine → Consensus.
8. Transaction Engine → Version Store.
9. Transaction Engine → Hybrid Clock.
10. Version Store → Hybrid Clock.
11. Consensus → Wire Protocol.
12. Consensus → Write-Ahead Log.
13. Client Library → Wire Protocol.
14. Wire Protocol → Metrics Registry.

Formula:

$$\text{Average coupling} = \frac{\text{Number of connections}}{\text{Number of modules}}$$

Substitution:

$$\text{Average coupling} = \frac{14}{13} = 1.0769 \approx 1.08$$

Answer: There are **14 production dependency connections**, **13 modules**, and an average of approximately **1.08 connections per module** under the convention used in the reference.

Meaning: The graph is relatively sparse: each module has only a small number of direct dependencies on average. A low average alone does not prove good design, but together with clear interfaces and one-way layering it is evidence of bounded coupling.

8.2 Numerical 2: Module instability

The reference defines instability as the proportion of a module's total incoming and outgoing dependencies that are outgoing. Let F_{in} be the number of modules that depend on the module, and F_{out} be the number of modules on which it depends.

$$I = \frac{F_{out}}{F_{in} + F_{out}}$$

When $I = 0$, the module has no outgoing dependencies and is maximally stable in this measure. When $I = 1$, it has no incoming dependencies and is maximally unstable in this measure.

8.2.1 Worked example A: Wire Protocol

Given $F_{in} = 2$ (Consensus and Client Library) and $F_{out} = 1$ (Metrics Registry):

$$I_{wire} = \frac{1}{2+1} = \frac{1}{3} \approx 0.33$$

Interpretation: Wire Protocol has two dependent modules and one dependency of its own, so its instability is approximately 0.33. Its envelope format should be changed conservatively because changes may affect both callers.

8.2.2 Worked example B: Transaction Engine

Given $F_{in} = 1$ (Web Layer) and $F_{out} = 3$ (Consensus, Version Store, Hybrid Clock):

$$I_{storage} = \frac{3}{1+3} = \frac{3}{4} = 0.75$$

Interpretation: The Transaction Engine has a higher instability value because it depends on several lower-level services while only the Web Layer depends directly on it. Its interface can evolve more freely than a heavily depended-upon primitive, although compatibility still needs to be managed.

8.2.3 Full instability table

Module	F_{in}	F_{out}	Instability I	Interpretation
CLI Client	0	1	1.00	Top-level front end
Sharding Server	0	2	1.00	Top-level gateway
Web Layer	0	3	1.00	Composition root
Transaction Engine	1	3	0.75	High-level orchestrator
Consensus	1	2	0.67	Mid-layer subsystem
Version Store	1	1	0.50	Balanced dependency profile
Client Library	2	1	0.33	Relatively stable shared component
Wire Protocol	2	1	0.33	Relatively stable shared component
Hybrid Clock	2	0	0.00	Stable primitive
Metrics Registry	2	0	0.00	Stable primitive
Write-Ahead Log	1	0	0.00	Stable primitive
Operational AI	1	0	0.00	Leaf module in this graph
Routing Ring	1	0	0.00	Leaf module in this graph

Interpretation of the full table: In this graph, upper-layer front ends have high instability, while primitives such as the Hybrid Clock and WAL have no outgoing dependencies and therefore have

$I = 0$. This matches the intended architecture: user-facing code can change without forcing low-level primitives to import or understand higher-level concepts. Instability is a structural indicator, not a complete quality score; stability must still be balanced with responsibility and testability.

9 Design Review Checklist

The following checklist answers every review item in the Lab 8 student submission template.

- ☐ **Every important requirement belongs to at least one module.** R1–R8 are mapped in the module identification table.
- ☐ **Every module has a clear purpose.** Each row states one main responsibility and its package path.
- ☐ **Diagram matches the module table.** The main dependency diagram shows the 13 production modules used in the numerical analysis; security and fault-injection support modules are explicitly identified separately.
- ☐ **Interfaces show inputs and outputs.** Four interfaces describe operations, parameters, results, and participating modules.
- ☐ **Unnecessary direct dependencies are avoided.** Narrow interfaces, opaque operations, layered imports, and a single composition root limit coupling.

10 Result

In this experiment, we designed the modules of `kvstore`. We identified **13 production modules** in the reference dependency graph and documented two additional supporting modules. We defined four interfaces between modules, improved cohesion by grouping each responsibility into a focused package, and reduced coupling through narrow interfaces, opaque consensus operations, and a layered dependency structure. The graph contains 14 production connections, giving an average of approximately 1.08 connections per module. The instability analysis further shows how the front-end layer can absorb change while low-level primitives remain stable.

Design takeaway

Final design principle: preserve one-way dependencies, keep core primitives small and reusable, and allow high-level modules to depend on narrow contracts rather than private implementation details.

11 References

1. *MCA SE Lab 8 Student Submission Template: Experiment 8 — Modular Design of a Software System*, supplied course template.
2. *Modular Design of a Software System Applied to the kvstore Distributed Key–Value Store*, supplied Lab 8 reference PDF, especially Sections 3–10 and Tables 1–12.
3. Neha Nupur, *Software Requirements Specification (SRS): kvstore — A Distributed, Fault-Tolerant, Transactional Key–Value Store*, Version 1.0, 2026. In particular, Sections 3–6 describe the architecture, module responsibilities, functional requirements, and non-functional constraints.

End of Lab Experiment 8